

StarPU - Fortran

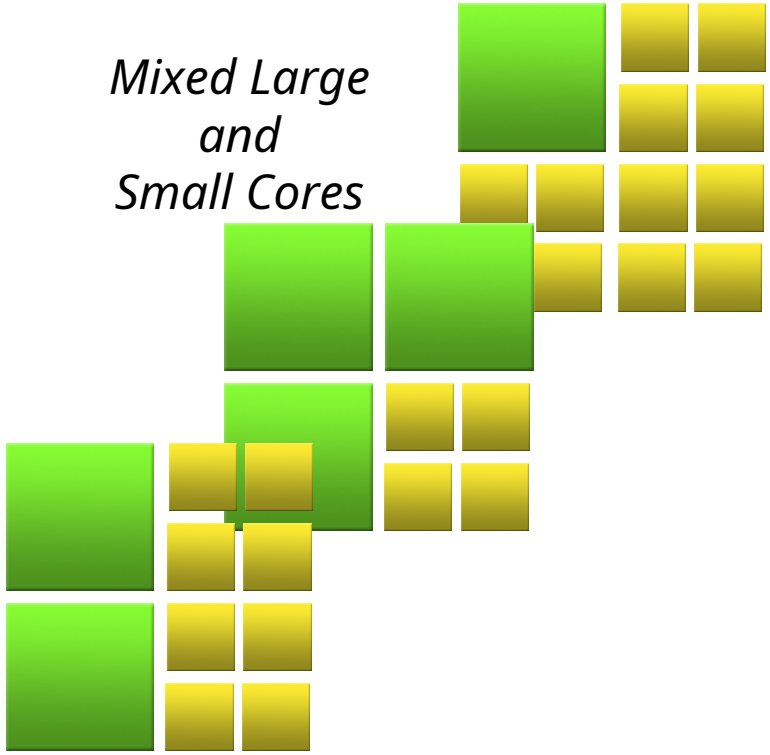
Nathalie Furmento - Samuel Thibault

INRIA Bordeaux - Sud-Ouest -- STORM Team

Introduction

Toward heterogeneous multi-core architectures

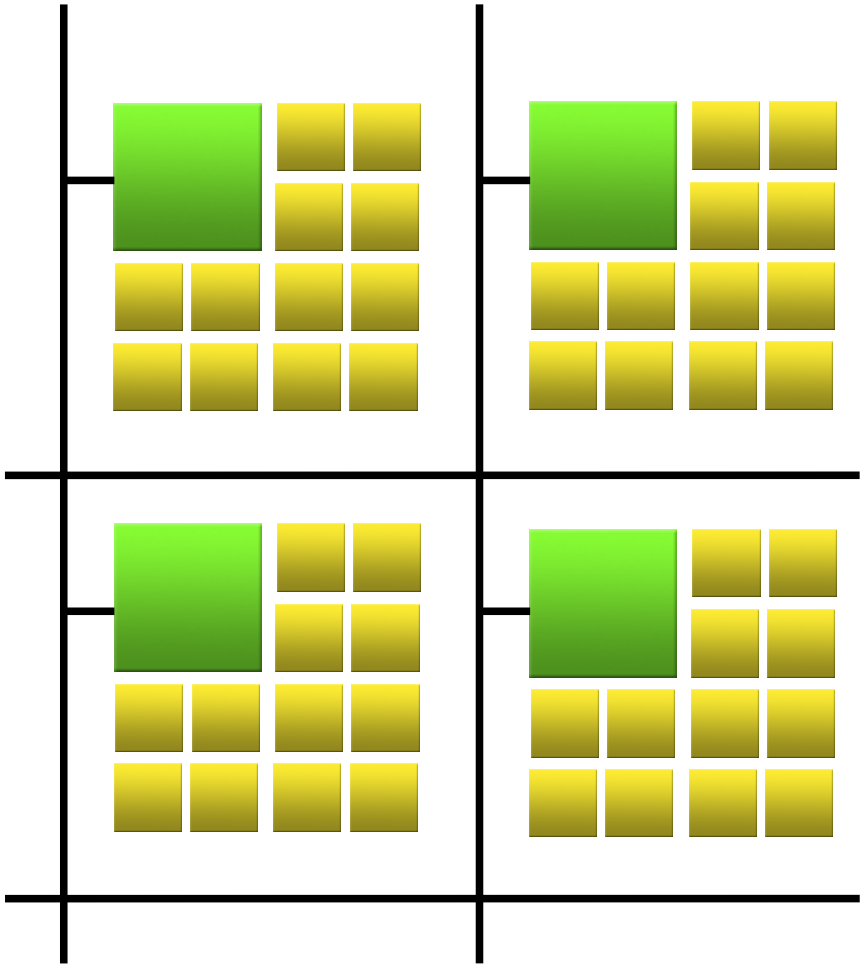
- Multicore is here
  - Hierarchical architectures
  - Manycore
  - Heterogeneous systems
- Architecture specialization
  - Now
    - Accelerators (GPGPUs, FPGAs)
    - Coprocessors (Xeon Phi)
    - All of the above
  - In the near Future
    - Many simple cores
    - A few full-featured cores



Introduction

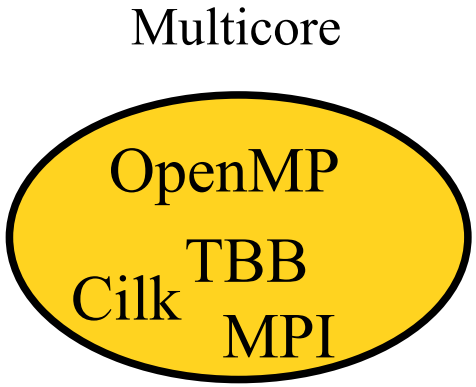
Toward heterogeneous multi-core clusters

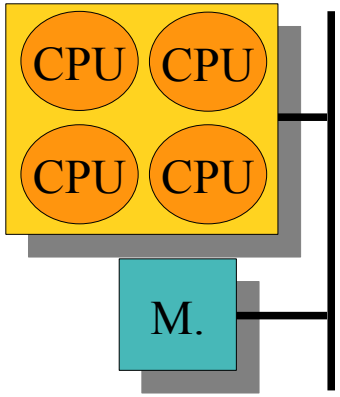
- Multicore is here
  - Hierarchical architectures
  - Manycore
  - Heterogeneous systems
- Clusters thereof
  - High-speed network
  - Network topology
  - Towards exascale



How to program these architectures?

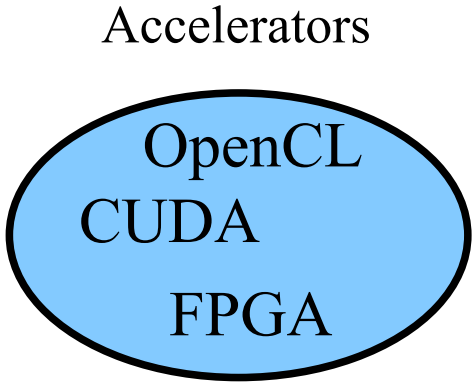
- Multicore programming
  - pthreads, OpenMP, TBB, ...

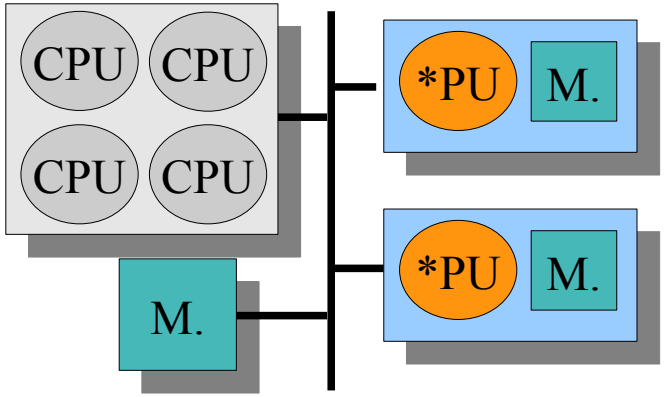




How to program these architectures?

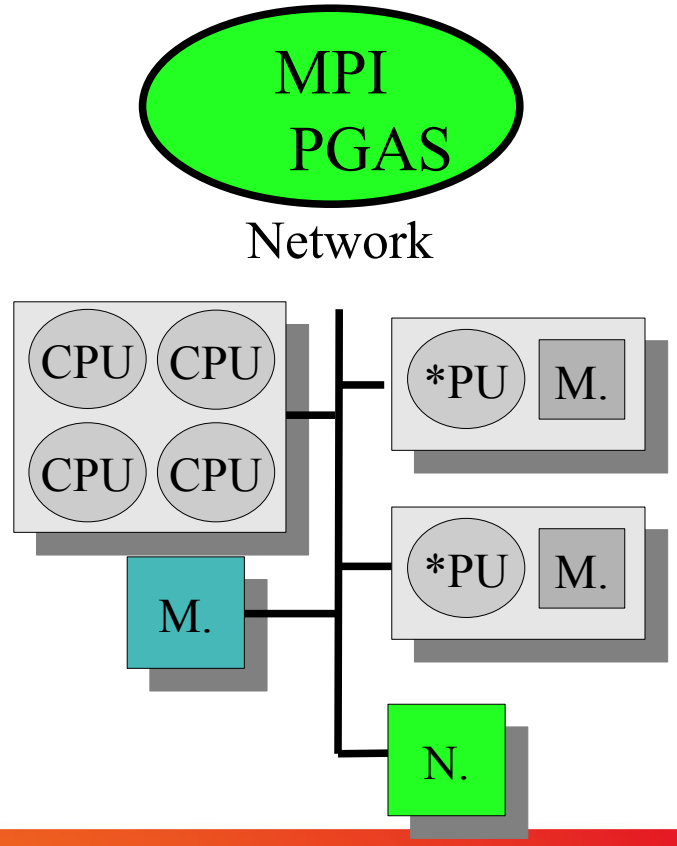
- Multicore programming
  - pthreads, OpenMP, TBB, ...
- Accelerator programming
  - CUDA, OpenCL, FPGA ?
  - OpenMP 5.0? Kokkos?
  - (Often) Pure offloading model





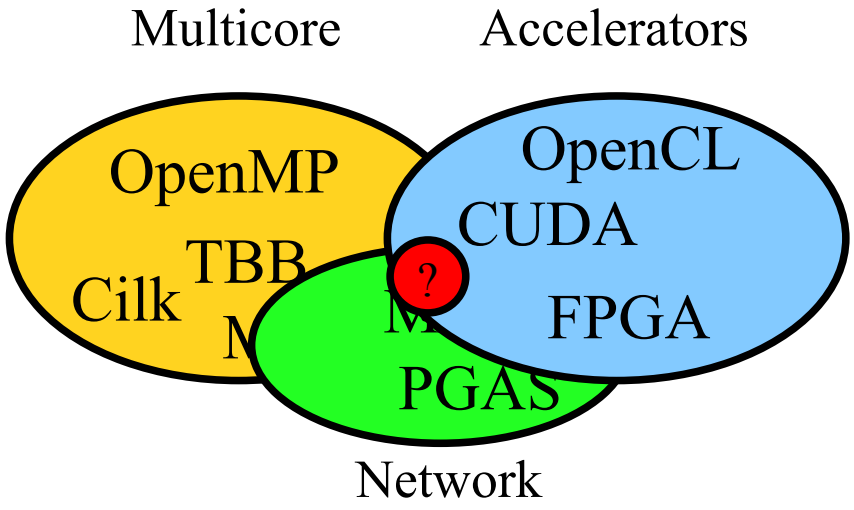
How to program these architectures?

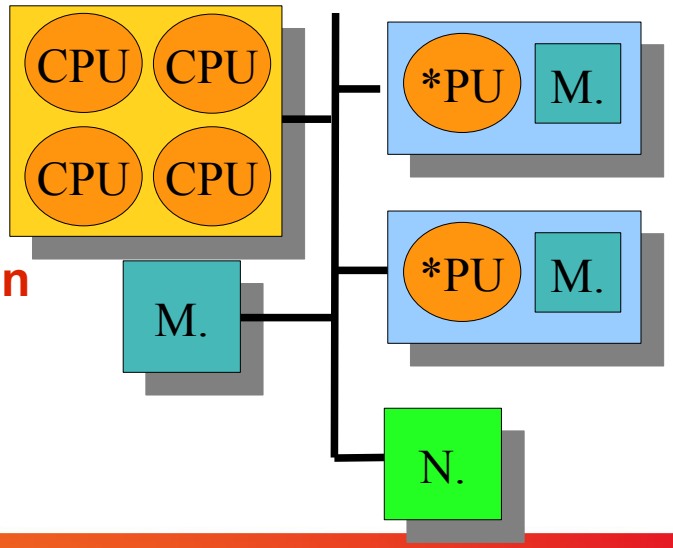
- Multicore programming
  - pthreads, OpenMP, TBB, ...
- Accelerator programming
  - CUDA, OpenCL, FPGA ?
  - OpenMP 5.0? Kokkos?
  - (Often) Pure offloading model
- Network support
  - MPI / PGAS



How to program these architectures?

- Multicore programming
  - pthreads, OpenMP, TBB, ...
- Accelerator programming
  - CUDA, OpenCL, FPGA ?
  - OpenMP 5.0? Kokkos?
  - (Often) Pure offloading model
- Network support
  - MPI / PGAS
- Hybrid models?
  - **Take advantage of all resources** ☺
  - **Complex interactions and distribution** ☹

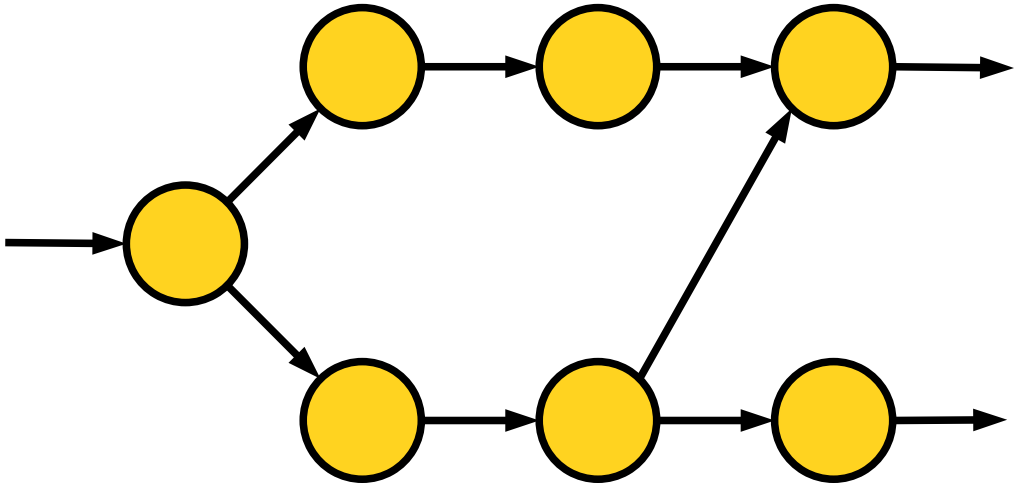




Task graphs

- Well-studied for scheduling parallelism (since 60's!)
- But only recent trend in HPC
- Departs from usual sequential programming

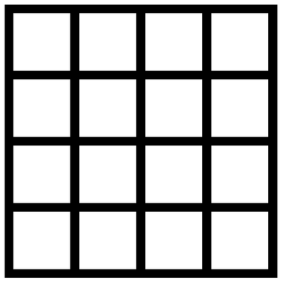
Really ?



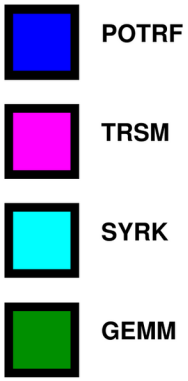
Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)



```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

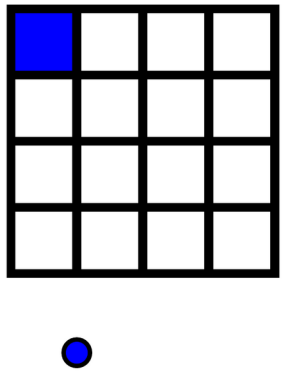


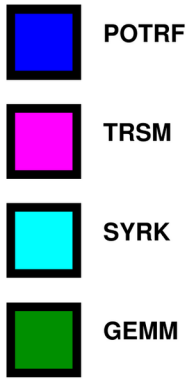
Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```



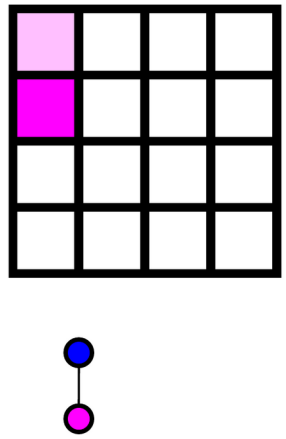


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

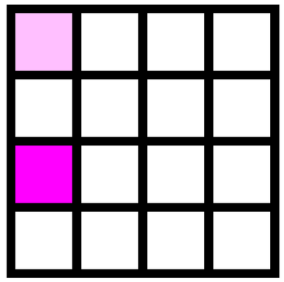


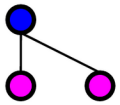
Expressing a task graph

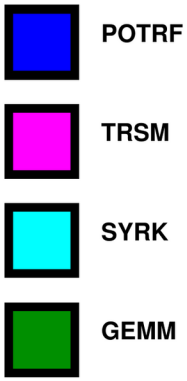
Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```





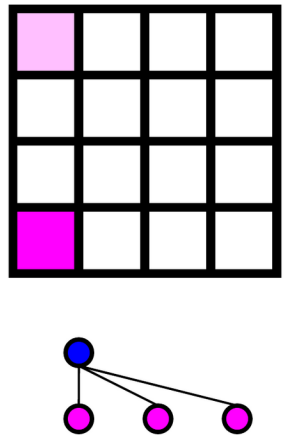


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

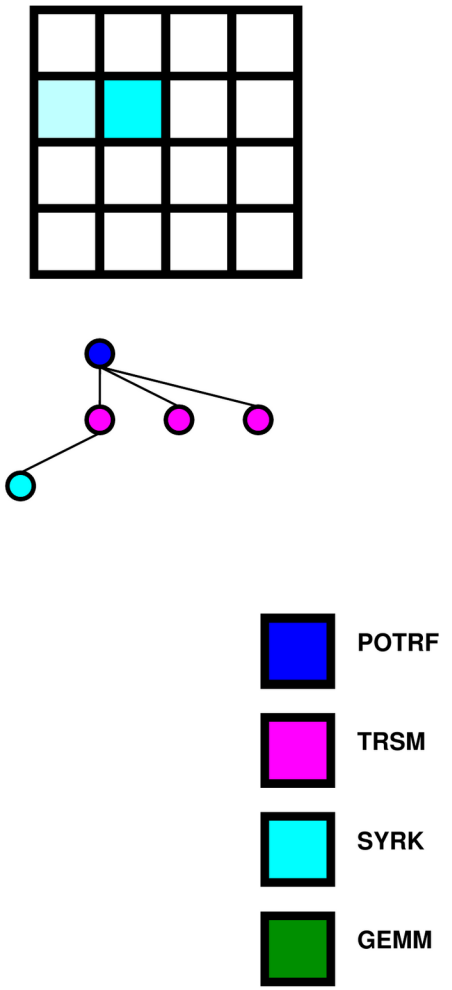


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

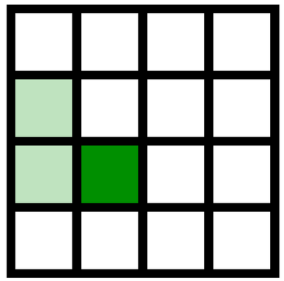


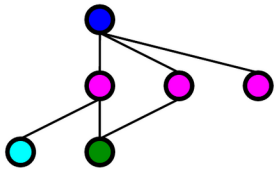
Expressing a task graph

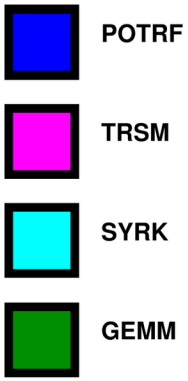
Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```





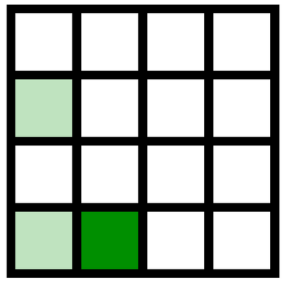


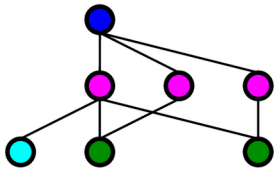
Expressing a task graph

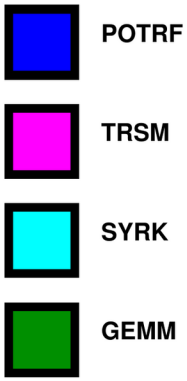
Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```





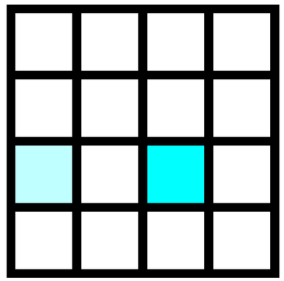


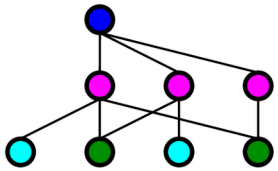
Expressing a task graph

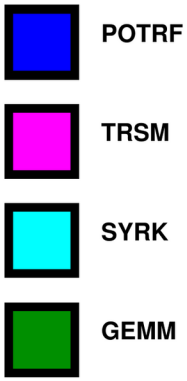
Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```





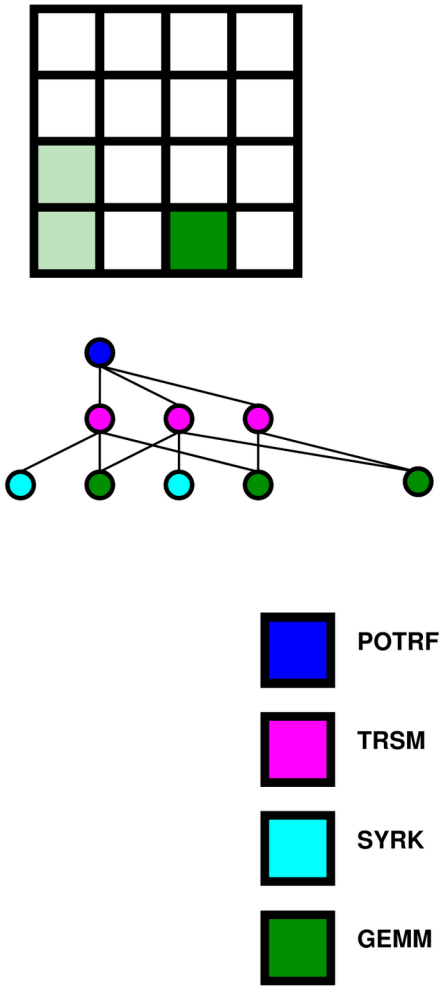


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

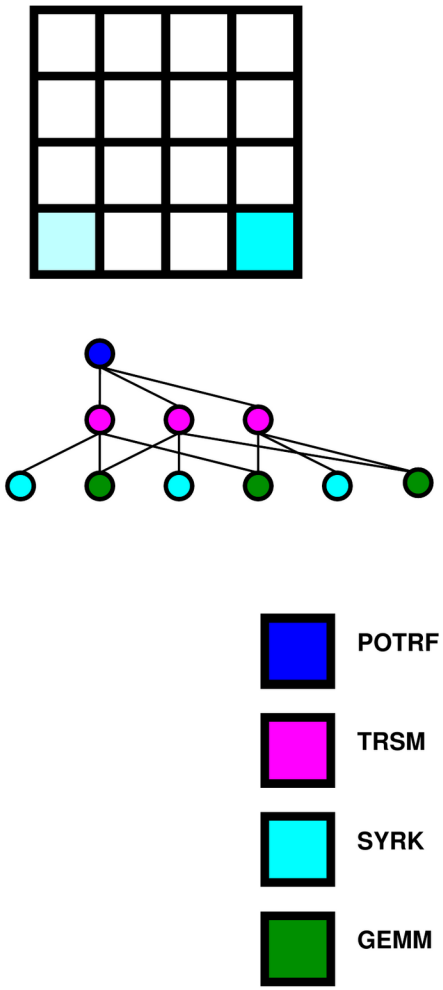


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

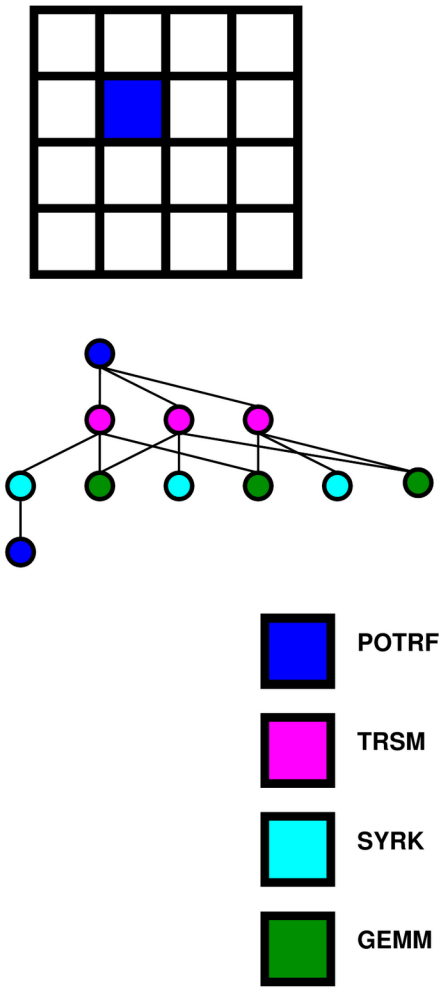


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```

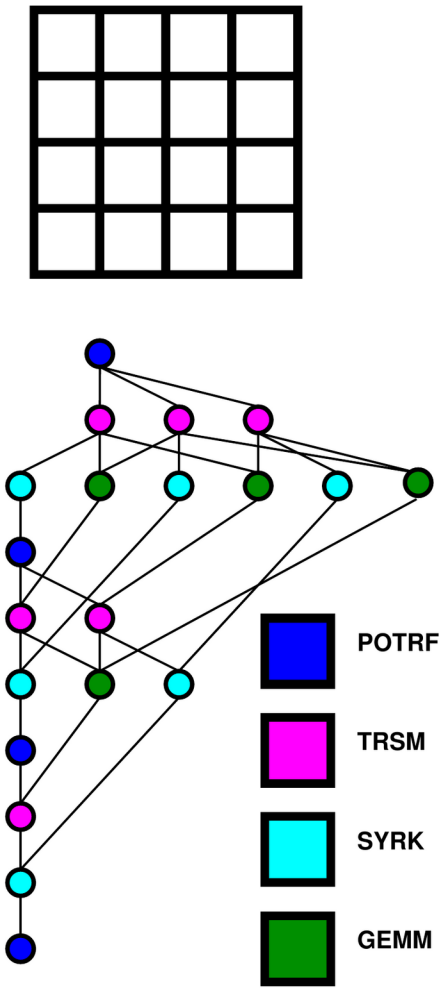


Expressing a task graph

Implicit task dependencies

- Right-Looking Cholesky decomposition (from PLASMA)

```
for (j = 0; j < N; j++) {
  POTRF (RW,A[j][j]);
  for (i = j+1; i < N; i++)
    TRSM (RW,A[i][j], R,A[j][j]);
  for (i = j+1; i < N; i++) {
    SYRK (RW,A[i][i], R,A[i][j]);
    for (k = j+1; k < i; k++)
      GEMM (RW,A[i][k],
            R,A[i][j], R,A[k][j]);
  }
}
task_wait_for_all();
```



Write your application as a task graph

Even if using a sequential-looking source code

➔ Portable performance

Sequential Task Flow (STF)

- Algorithm remains the same on the long term
- Can debug the sequential version.
- Only kernels need to be rewritten
  - BLAS libraries, multi-target compilers
- Runtime will handle parallel execution

Task-based programming

- Needs code restructuring
  - Split computation into tasks
    - BLAS, typically
    - Supposed to have “stable” performance
- Constraining
  - No global variables
    - Mandatory for GPUs
- Actually… functional programming

So a good move, in the end ☺

- Have to accept constraints and losing control

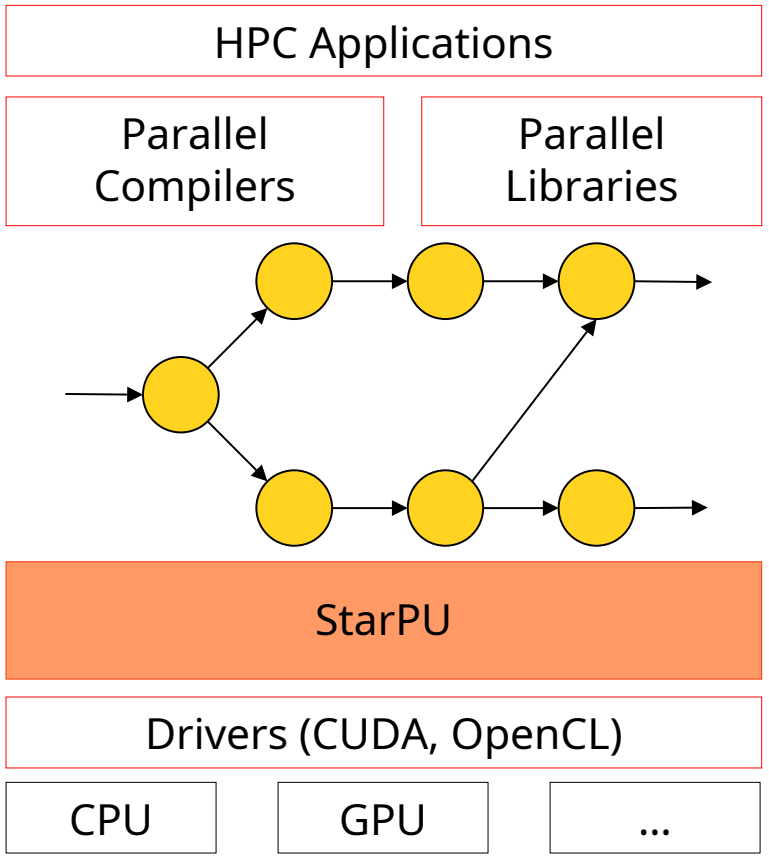
Just like we did when moving from assembly to high-level languages

Overview of StarPU

The StarPU runtime system

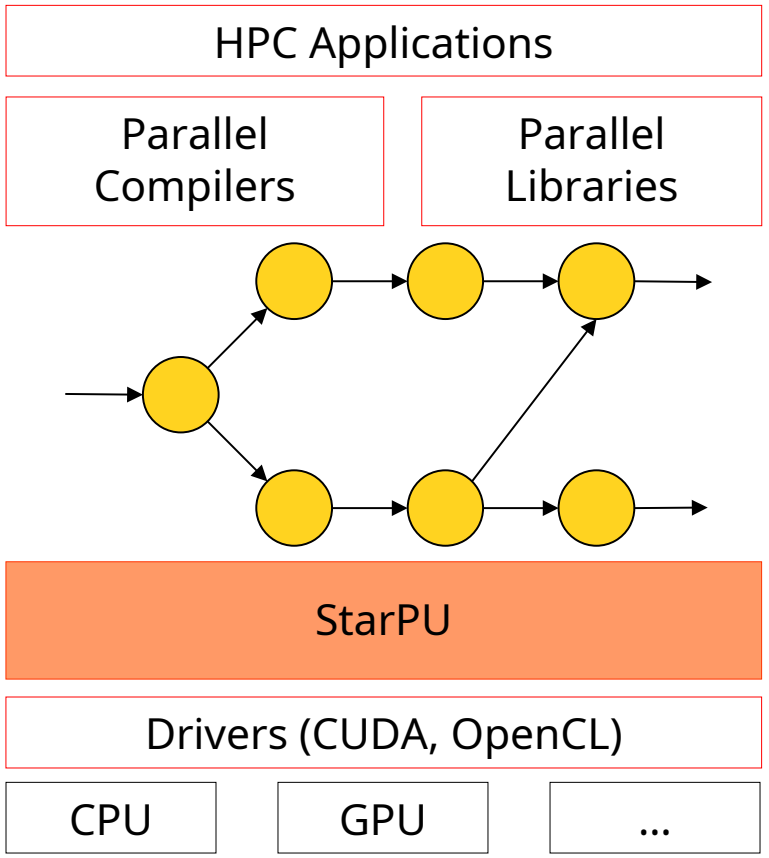
The need for runtime systems

- "do dynamically what can't be done statically anymore"
- Compilers and libraries generate (graphs of) tasks
  - Additional information is welcome!
- StarPU provides
  - Task scheduling
  - Memory management



Data management

- StarPU provides a Virtual Shared Memory (VSM) subsystem (aka DSM)
  - Replication
  - Consistency
  - Single writer
    - Or reduction, ...
- Input & ouput of tasks = reference to VSM data

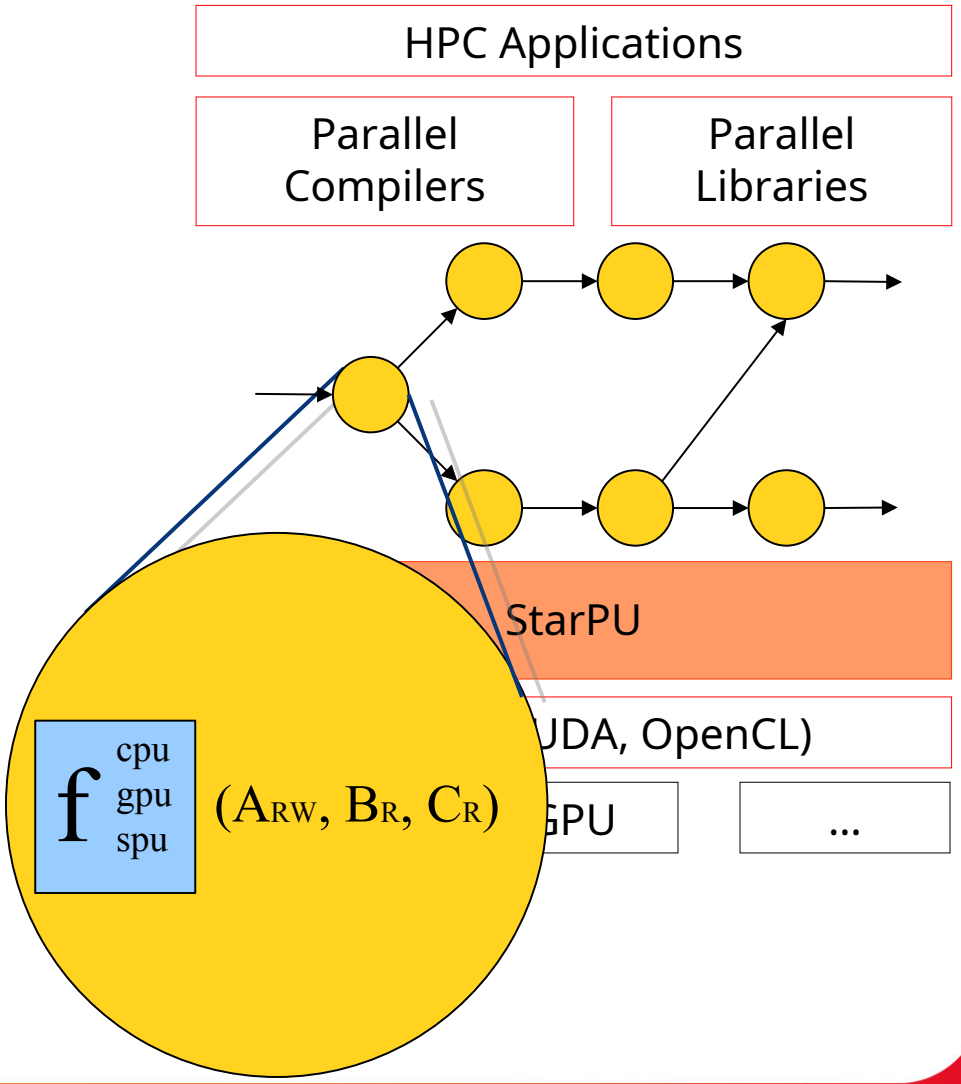


The StarPU runtime system

Task scheduling

- Tasks =
  - Data input & output
    - Reference to VSM data
  - Multiple implementations
    - E.g. CUDA + CPU implementation
  - Non-preemptible
  - Dependencies with other tasks

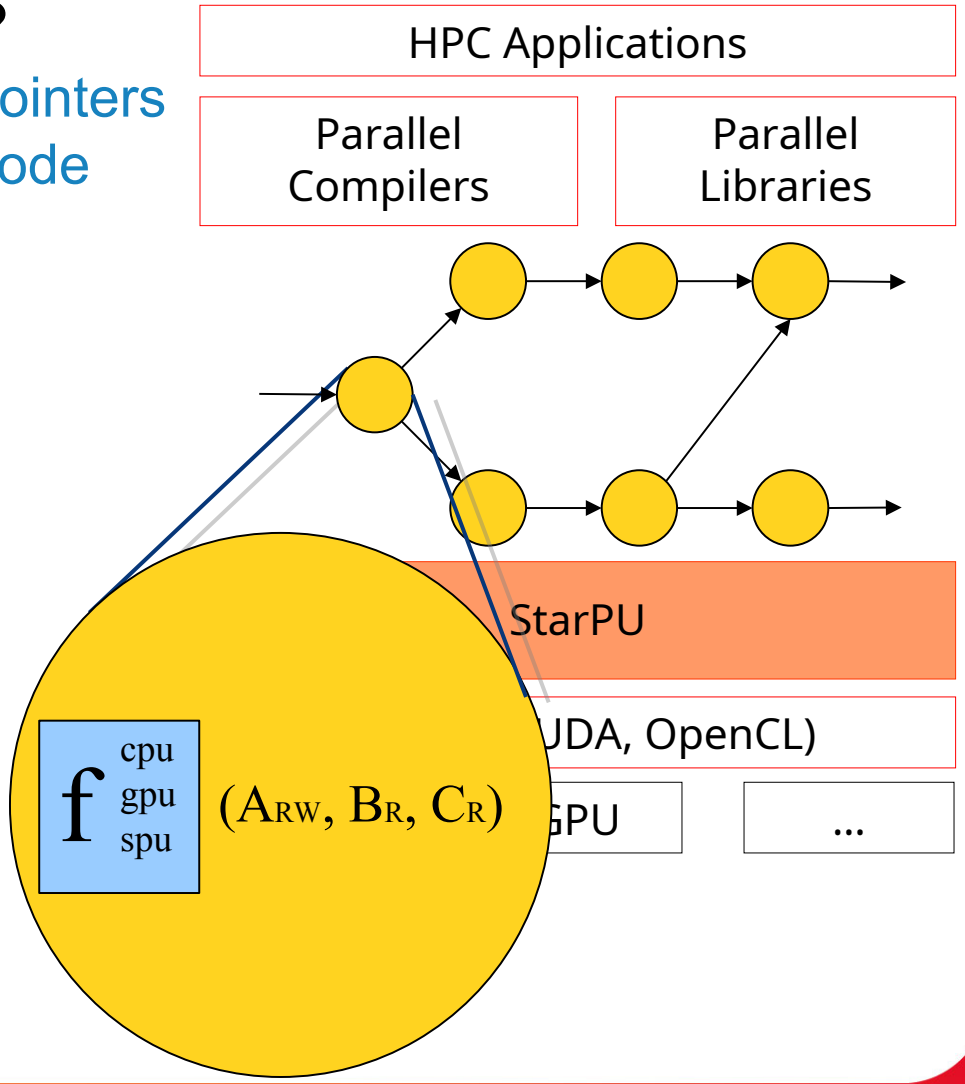
- StarPU provides an Open Scheduling platform
  - Scheduling algorithm = plug-ins



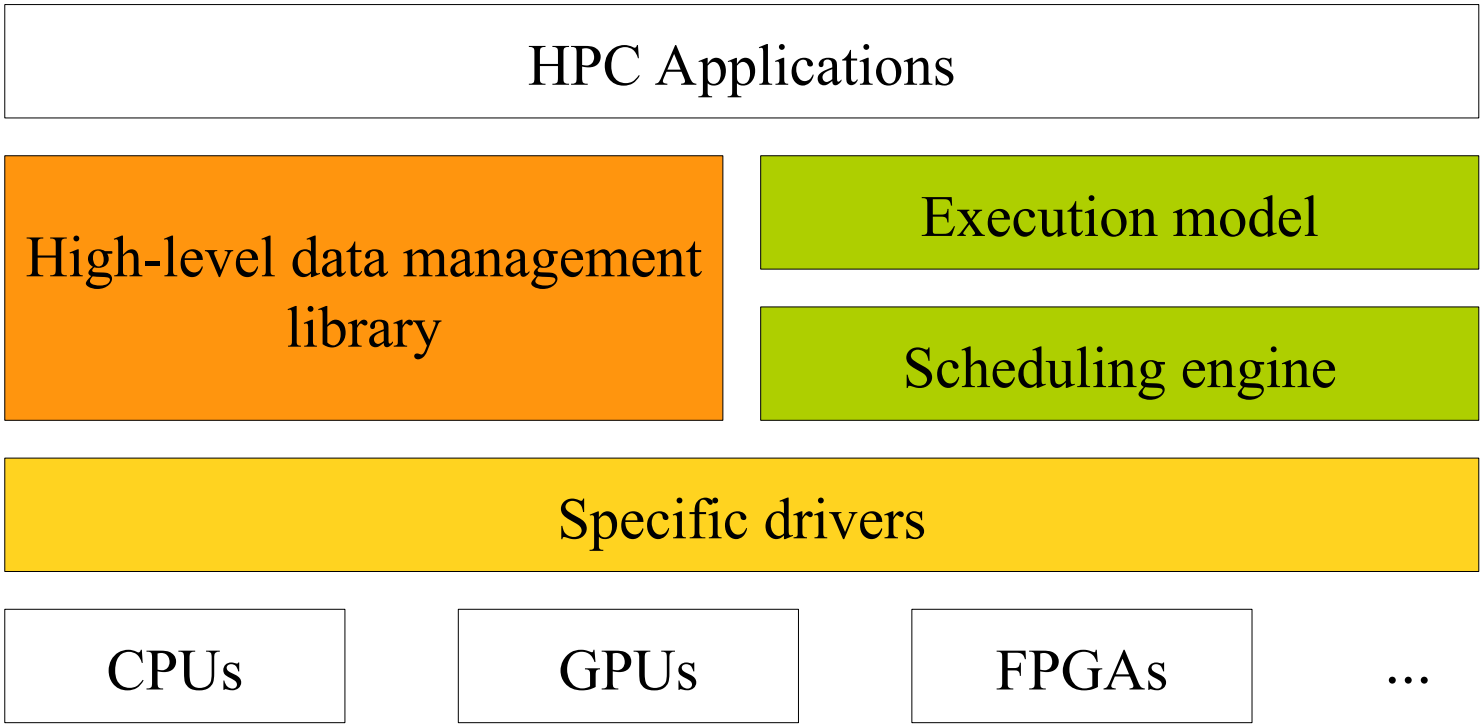
The StarPU runtime system

Task scheduling

- Who generates the code ?
  - StarPU Task ~= function pointers
  - StarPU doesn't generate code
- Libraries era
  - PLASMA + MAGMA
  - FFTW + CUFFT…
  - Variants management
- Rely on compilers
  - E.g. Kokkos



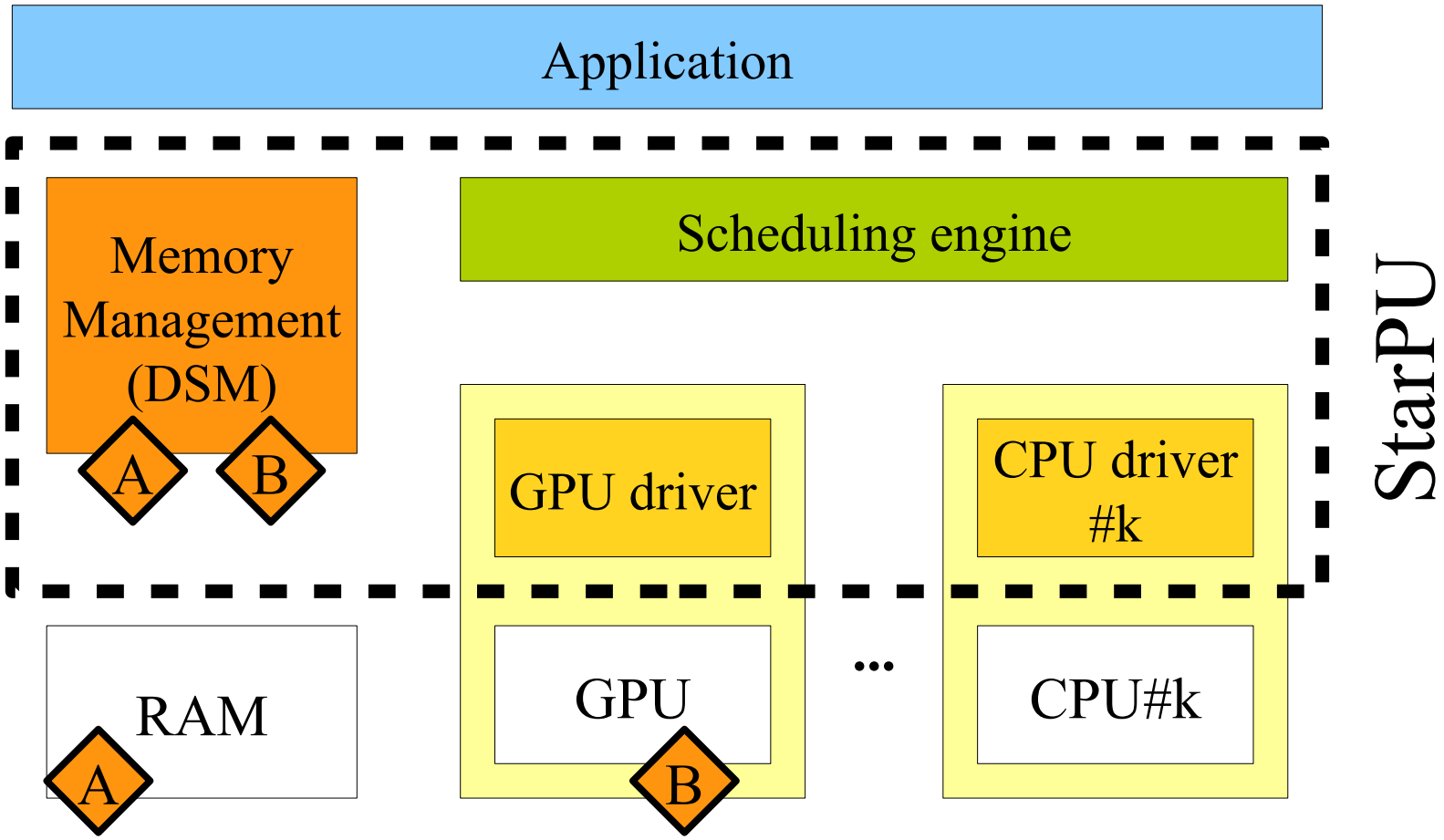
The StarPU runtime system



Mastering CPUs, GPUs, FPGAs … ***PUs → StarPU**

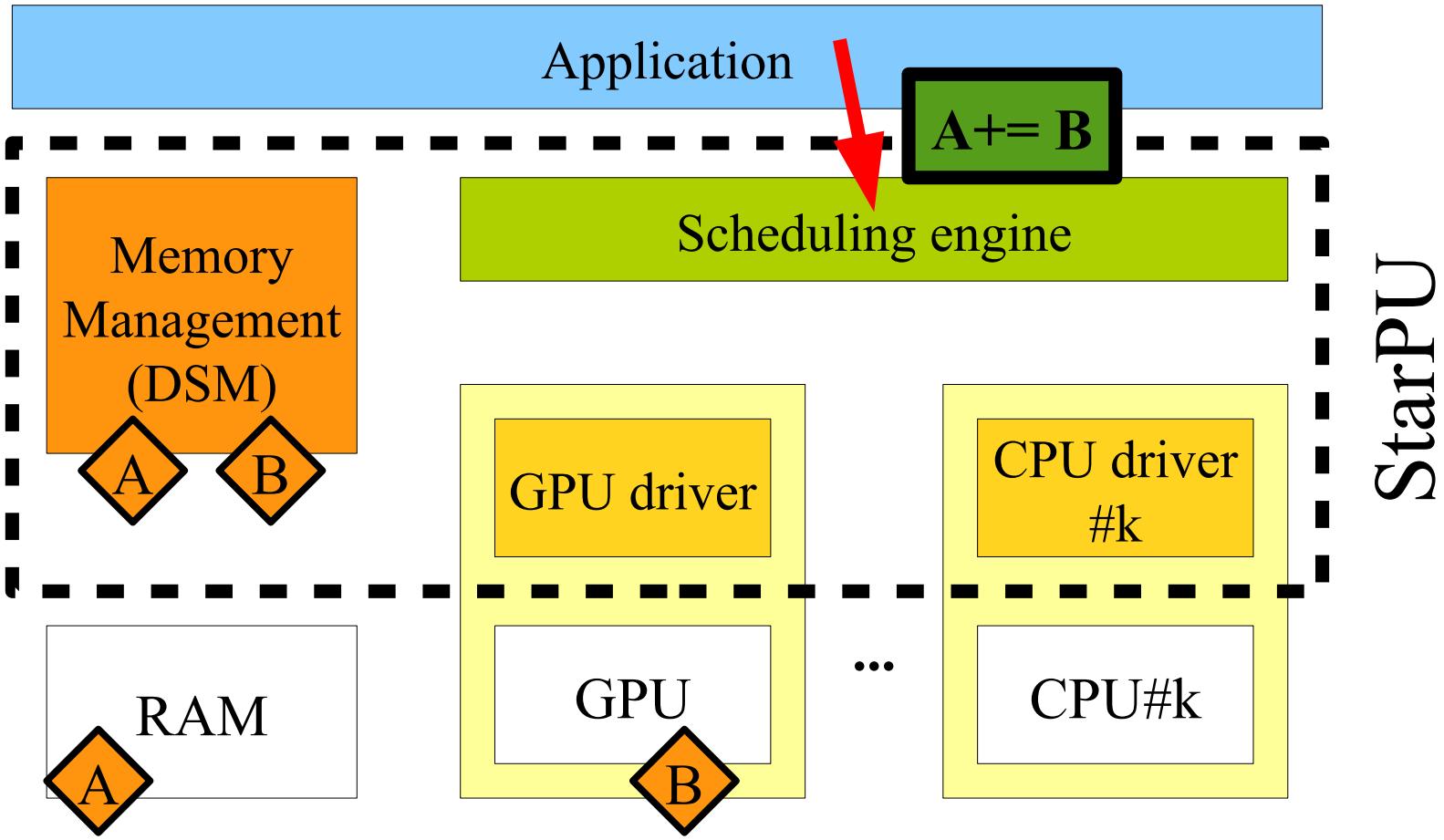
The StarPU runtime system

Execution model



The StarPU runtime system

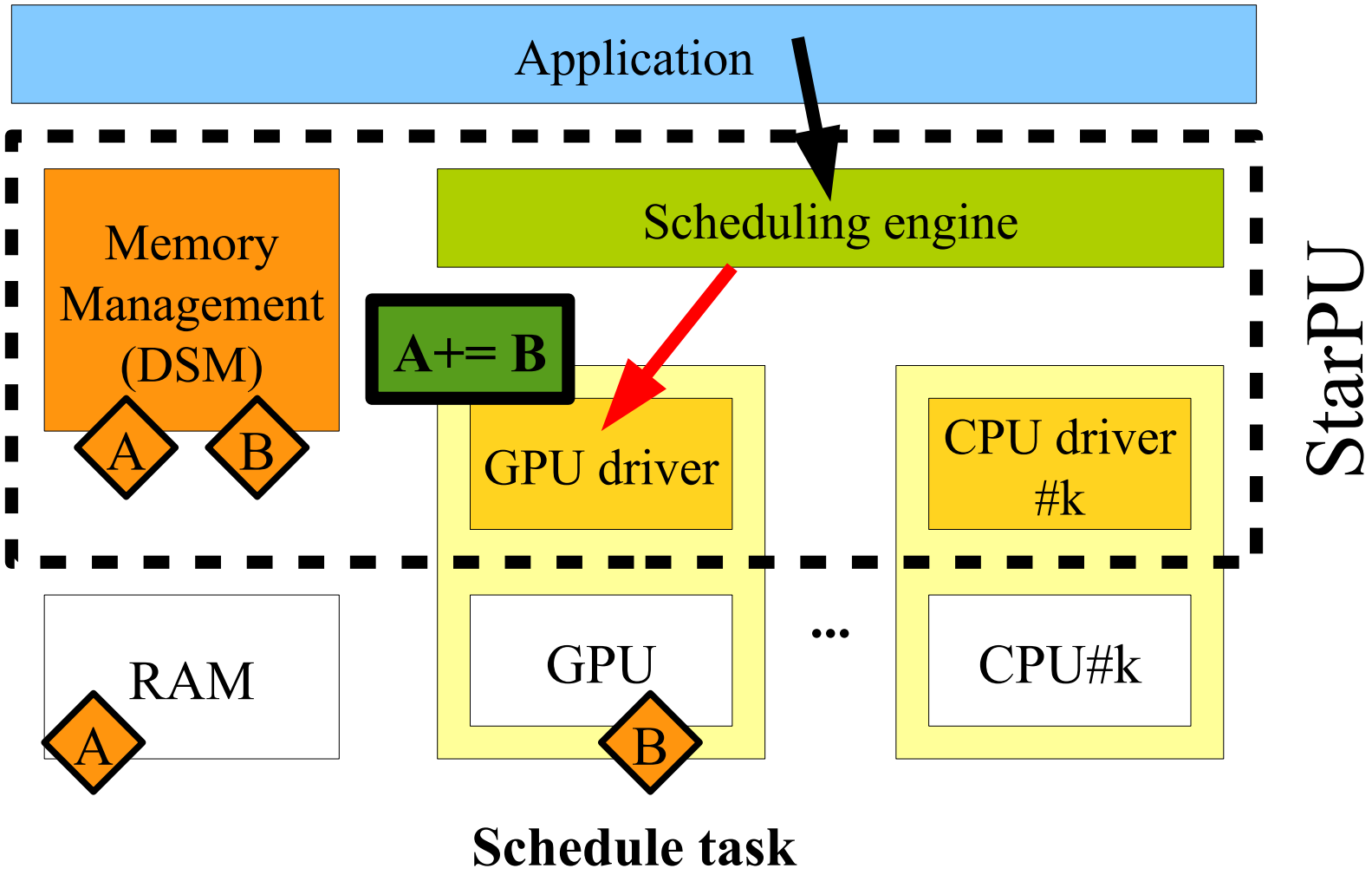
Execution model



Submit task « A += B »

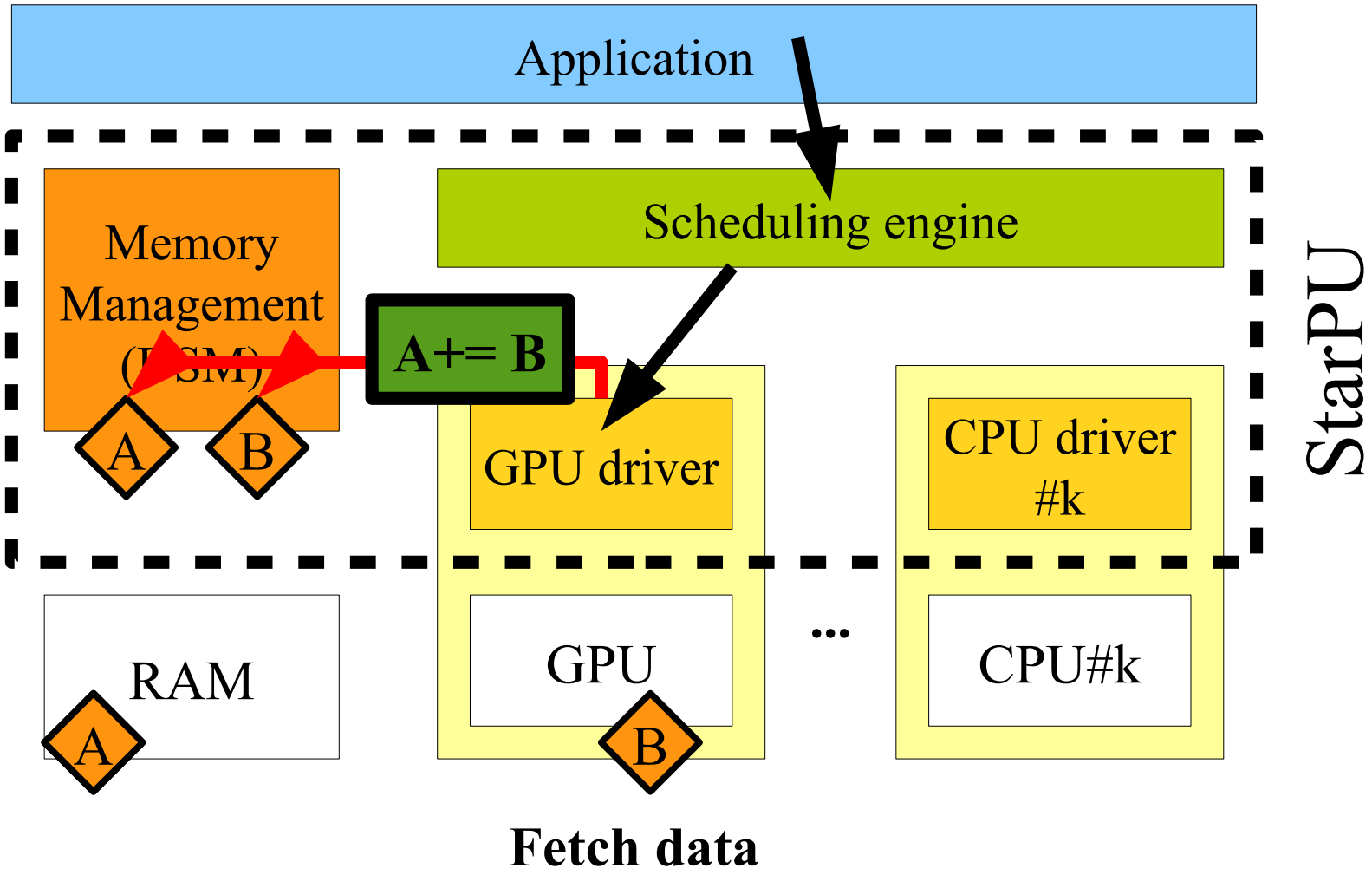
The StarPU runtime system

Execution model



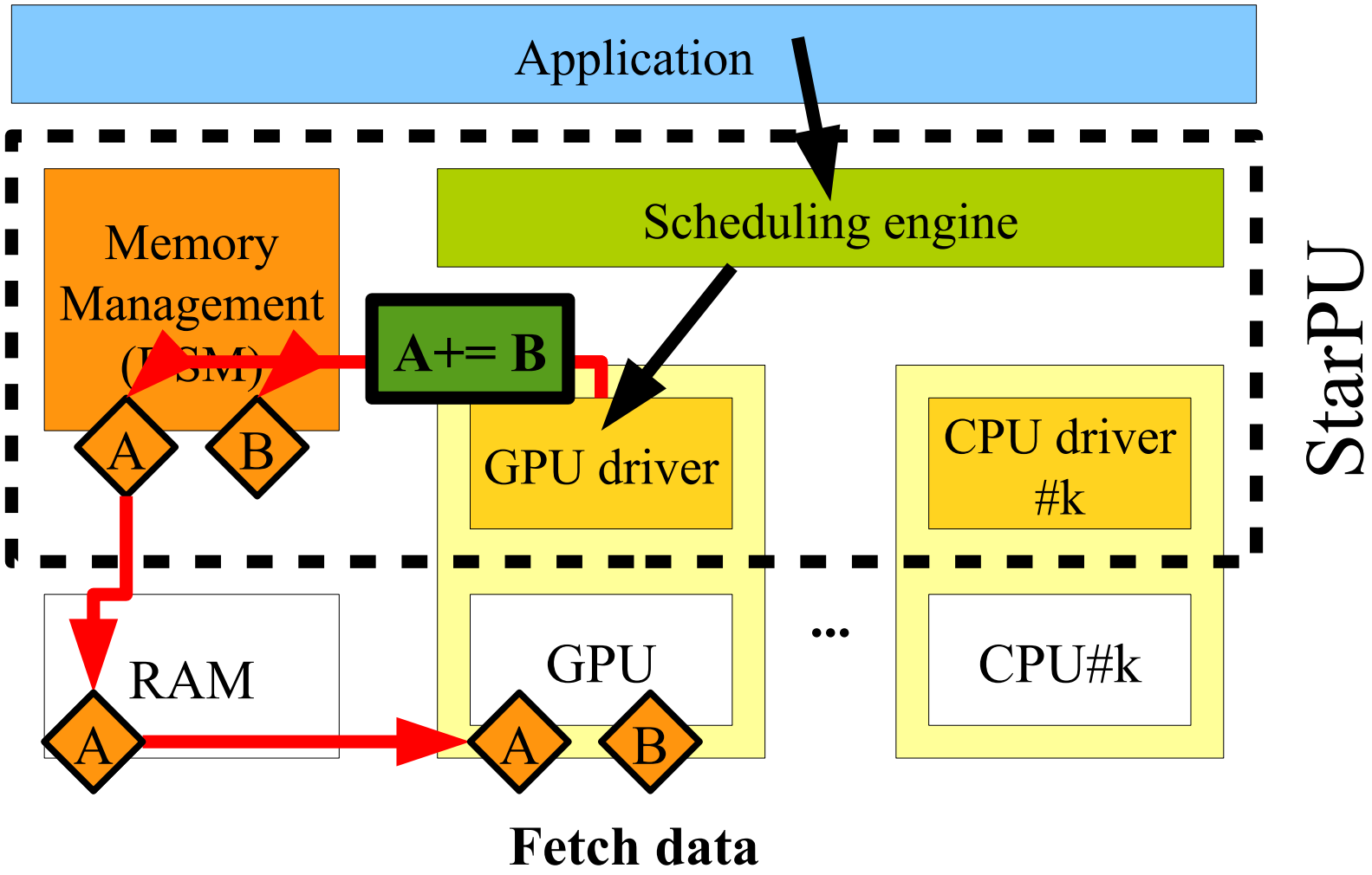
The StarPU runtime system

Execution model



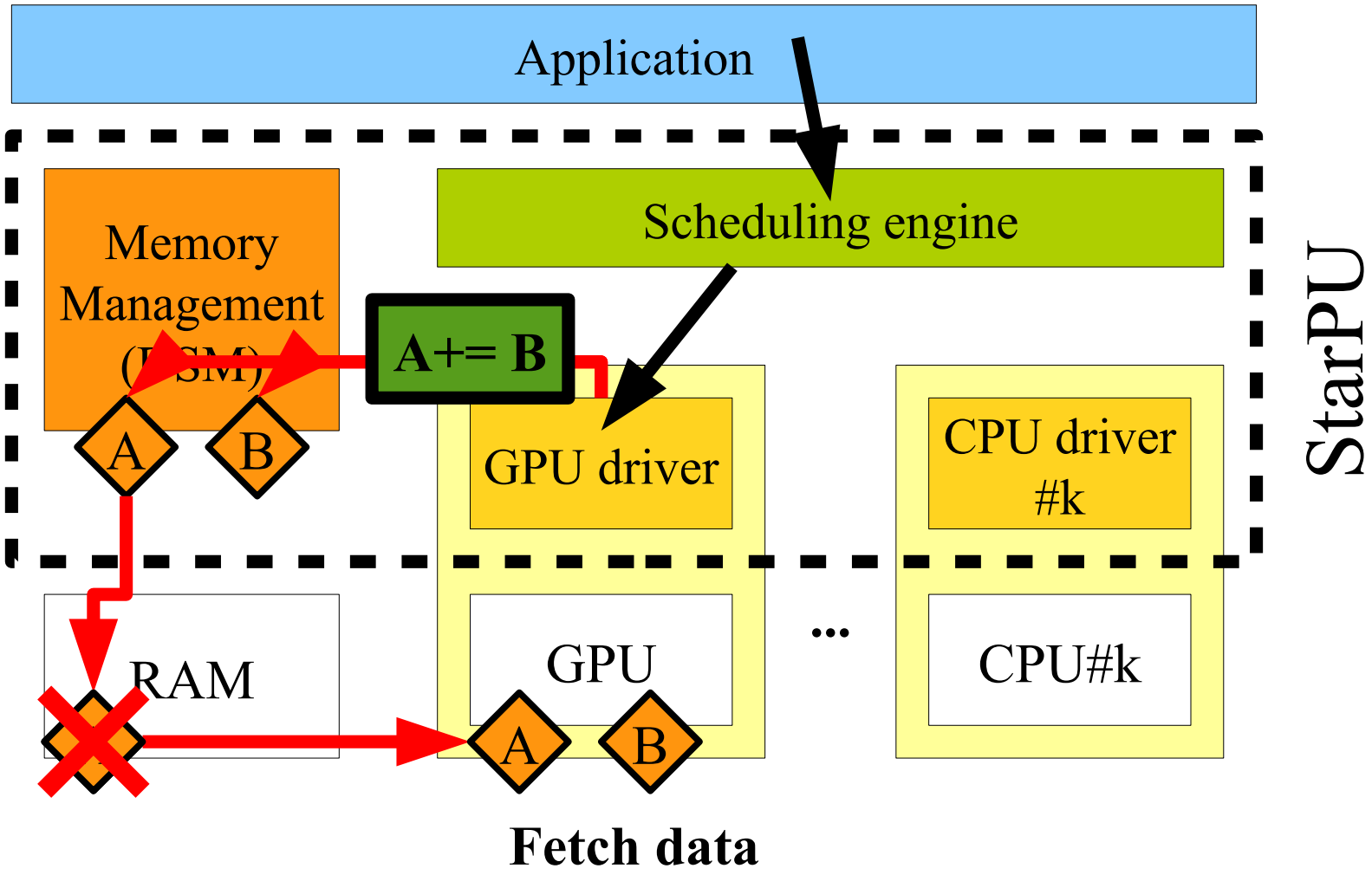
The StarPU runtime system

Execution model



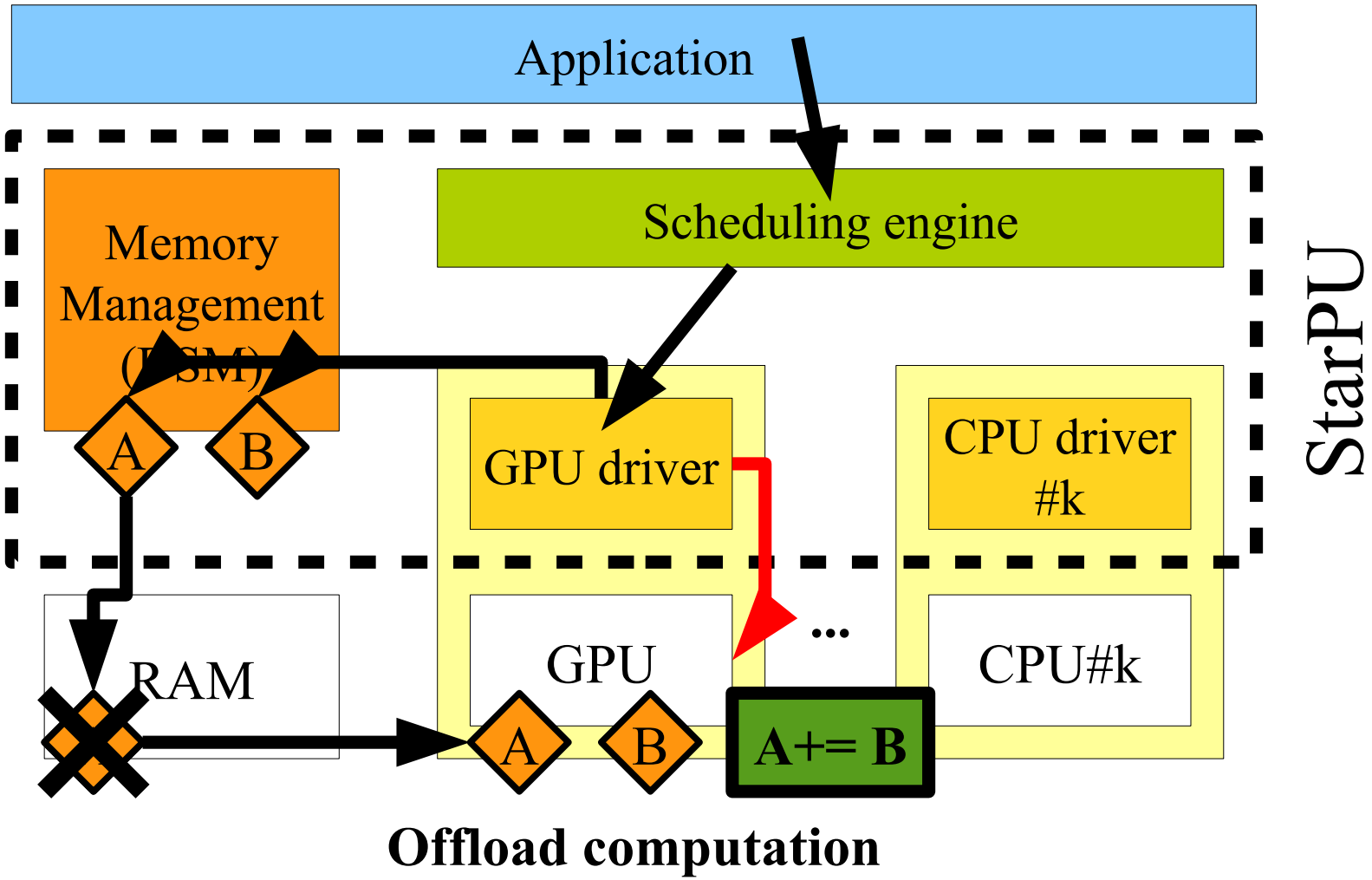
The StarPU runtime system

Execution model



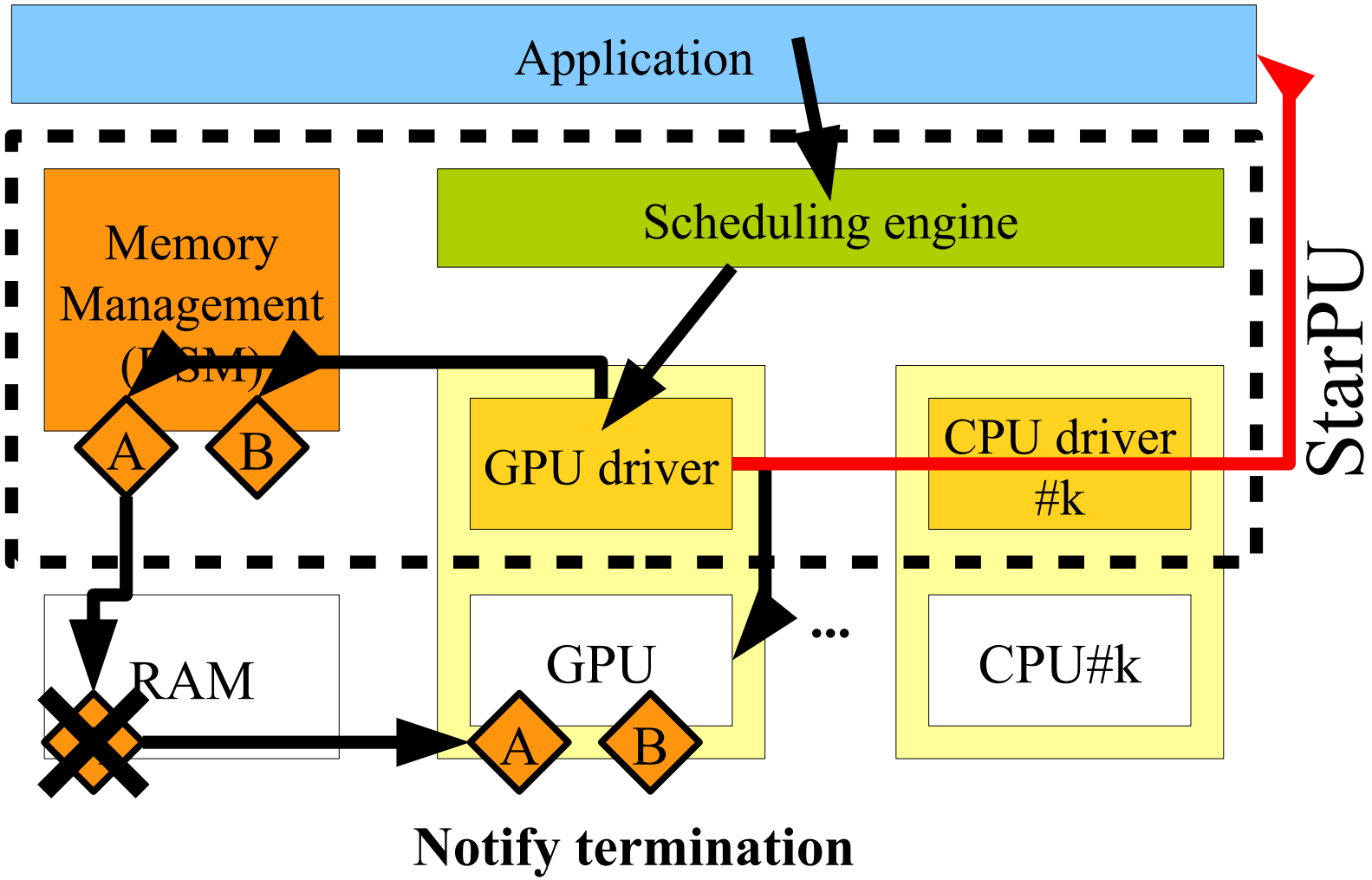
The StarPU runtime system

Execution model



The StarPU runtime system

Execution model



The StarPU runtime system

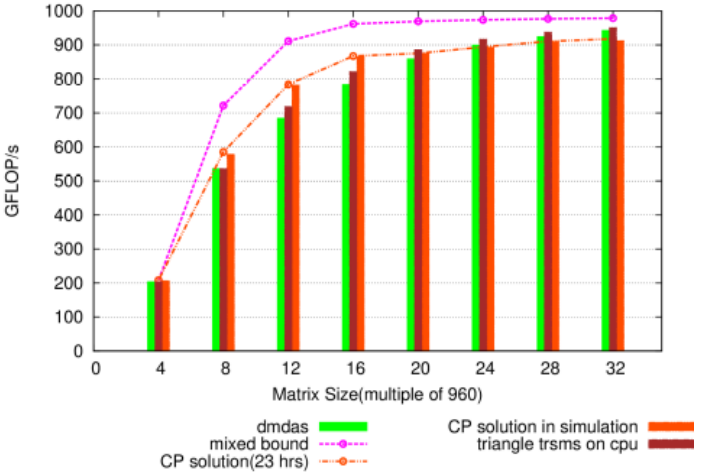
Development context

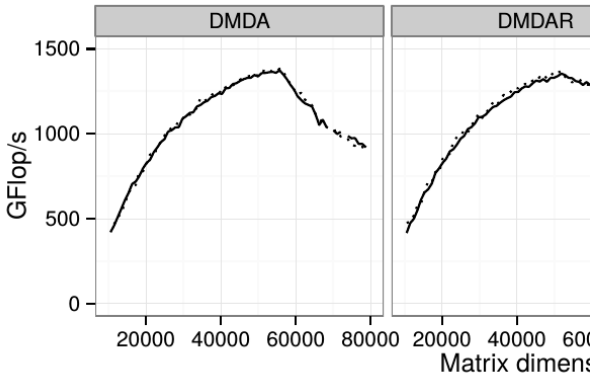
- History
  - Started in 2008
    - PhD Thesis of Cédric Augonnet
  - StarPU main core ≈ 70k lines of code
  - Written in C
- Open Source
  - Released under LGPL
  - Sources freely available
    - git repository and nightly tarballs
    - See https://starpu.gitlabpages.inria.fr/
  - Open to external contributors
- [HPPC'08]
- [Europar'09] – [CCPE'11],... >2000 citations

The StarPU runtime system

Task-based support

- Task/data scheduling
  - Pipelining
  - Load balancing
  - GPU memory limitation management
  - Data prefetching
- Performance bounds
- Distributed execution
- High-level performance analysis
- Out-of-core: optimized swapping to disk
- Debugging sequential execution
- Reproducible performance simulation





Terminology

- Data handle
  - Designates data managed by StarPU
  - input/output for tasks
- Codelet
  - Gathers different implementation variants achieving the same computation
  - Instantiated as tasks
- Task
  - Relates the two
  - Instantiation of a codelet to be run over data handles
- Worker
  - Executes tasks

Dependencies

- Implicit dependencies inferred from RaW/WaW/WaR flow dependencies
  - Sequential Task Flow STF
- Explicit dependencies
- Explicit dependencies between tag numbers
  - Rendez-vous points
- On-the-fly submission of tasks
  - Dependencies managed by application

Implicit dependencies

Inferred from RaW/WaW/WaR flow dependencies

- Sequential Task Flow

```
starpu_task_insert(&cl, STARPU_RW, vector_handle, 0);
starpu_task_insert(&cl, STARPU_RW, vector_handle, 0);
```

Nice for end-user source code

Also more advanced dependencies

```
starpu_task_insert(&cl, STARPU_RW|STARPU_COMMUTE, vector_handle, 0);
starpu_task_insert(&cl, STARPU_RW|STARPU_COMMUTE, vector_handle, 0);
```

Explicit dependencies

```
task1 = starpu_task_build(&cl, STARPU_RW, A_handle, 0);
task2 = starpu_task_build(&cl, STARPU_RW, B_handle, 0);
starpu_task_declare_deps(task2, 1, task1);
starpu_task_submit(task1);
starpu_task_submit(task2);
```

Here separated building and submitting task2

- To declare dependency before submission
  - Otherwise would start immediately

Can also be a parameter of starpu_task_build

Nice for libraries

Explicit tag dependencies

```
starpu_tag_declare_deps((starpu_tag_t) 1235, 1,
(starpu_tag_t)1234);

starpu_task_insert(&cl, STARPU_RW, A_handle, STARPU_TAG,
(starpu_tag_t) 1234, 0);

starpu_task_insert(&cl, STARPU_RW, B_handle, STARPU_TAG,
(starpu_tag_t) 1235, 0);
```

Can introduce synchronization with a task that was not built yet

- Rendez-vous points

Nice for parametric task graphs

Interaction with StarPU execution

- Can wait for a given task

```
starpu_task_wait(task);
```

- Can access the result while computing

```
starpu_task_insert(&cl, …);
starpu_data_acquire(vector_handle, STARPU_R);
printf("%d", array[0]);
starpu_data_release(vector_handle);
```

- Or as a callback

```
starpu_task_insert(&cl, …);
starpu_data_acquire_cb(vector_handle, STARPU_R,
test_converged, NULL)
```

- And many more

Comments on pragma-based approaches

```
#pragma task(A)
  { ... }
#pragma task(B) depends(A)
  { ... }
```

- Programmability?

```
#pragma task output(A[i])
  { ... }
#pragma task input(A[i])
  { ... }
```

- Like Sequential Task Flow
  - Can be dumb rendez-vous points
- Path to accelerator support (but also needs size of data)

Applications on top of StarPU

Using CPUs, GPUs, distributed, out of core, ...

- Dense linear algebra
  - Cholesky, QR, LU, ... : Chameleon (based on Plasma/Magma)
- Sparse linear algebra
  - QR_MUMPS
  - PaStiX
- Compressed linear algebra
  - BLR, h-matrices
- Fast Multipole Method
  - ScalFMM
- Conjugate Gradient
- Other programming models : Data flow, skeletons
  - SignalPU, SkePU
- ...

Conclusion

Summary

Task API

- Be flexible
  - Several ways to express dependencies
- Be convenient
  - Concise API
- Or not
  - Be programmable
  - Language-only APIs can pose problems

